



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Detección de Rostros con Visión Artificial

Alumnos:

Lara Beltran Jorge Alberto

Pacheco Perez Diego

Docente: Zuriel Dathan Mora Felix

Materia: Tópicos Selectos de Inteligencia Artificial

1. Introducción

El reconocimiento facial es una técnica de visión por computadora que permite identificar o verificar la identidad de una persona a partir de características biométricas de su rostro. En este proyecto se desarrolló un sistema de reconocimiento facial en tiempo real utilizando DeepFace, OpenCV, TensorFlow y SQLite.

El sistema permite detectar rostros mediante una cámara web, identificar estudiantes registrados en una base de datos de imágenes y mostrar información académica almacenada en una base de datos SQLite.

2. Objetivo

Desarrollar un sistema de reconocimiento facial capaz de:

- Detectar rostros en tiempo real.
- Identificar estudiantes previamente registrados.
- Consultar información académica desde una base de datos SQLite.
- Mostrar métricas de rendimiento como FPS y tiempo de reconocimiento.
- Evaluar el desempeño del modelo mediante Accuracy, Precision, Recall y F1-Score.

3. Tecnologías Utilizadas

Tecnología	Función
Python	Lenguaje principal
OpenCV	Captura de video y detección facial
DeepFace	Reconocimiento facial
FaceNet	Modelo de extracción de características faciales
TensorFlow	Soporte para procesamiento mediante GPU
SQLite	Almacenamiento de información de estudiantes
Albumentations	Aumento de datos

4. Arquitectura del Sistema

El sistema está compuesto por cuatro módulos principales:

1. Base de datos

Archivo:

crear_bd.py

Se crea una base de datos SQLite llamada:

estudiantes.db

La tabla almacena:

- Nombre
- ID del estudiante
- Edad
- Carrera
- Semestre

Ejemplo:

Nombre	ID	Edad	Carrera	Semestre
Jorge	21170363	22	Ing en sistemas	10

2. Aumento de Datos (Data Augmentation)

Archivo:

dataAugmentation.py

Con el objetivo de incrementar la cantidad y diversidad de imágenes disponibles para cada persona, se aplican transformaciones mediante Albumentations.

Transformaciones utilizadas:

Transformación	Objetivo
HorizontalFlip	Simular cambios de orientación
RandomBrightnessContrast	Variaciones de iluminación
Rotate	Simular inclinaciones de la cabeza
GaussNoise	Robustez ante ruido
Blur	Simular desenfoque

Por cada imagen original se generan: 5 imágenes aumentadas

Esto mejora la capacidad de generalización del modelo.

3. Sistema de Reconocimiento Facial

Archivo:

main.py

El sistema sigue el siguiente flujo:

Captura de Video

La cámara captura imágenes en tiempo real utilizando OpenCV.

Resolución:

640 x 480

FPS configurados:

30 FPS

Detección de Rostros

Se utiliza el clasificador Haar Cascade:

haarcascade_frontalface_default.xml

Parámetros:

scaleFactor=1.1

minNeighbors=5

minSize=(30,30)

Una vez detectado el rostro se dibuja un rectángulo alrededor del mismo.

Reconocimiento Facial

El reconocimiento se realiza mediante:

DeepFace.find()

Configuración:

model_name = "Facenet"

distance_metric = "cosine"

detector_backend = "opencv"

FaceNet genera un vector de características (embedding) para cada rostro y posteriormente compara dicho vector contra las imágenes almacenadas en la base de datos facial.

Consulta de Información

Cuando una persona es reconocida:

1. Se obtiene su nombre.
2. Se consulta la base SQLite.
3. Se recuperan sus datos académicos.

Información mostrada:

- Nombre
- ID
- Edad
- Carrera
- Semestre

Optimización

Para evitar procesar cada cuadro de video:

INTERVALO_RECONOCIMIENTO = 2 segundos

El reconocimiento solo se ejecuta cada dos segundos.

Esto reduce significativamente el consumo computacional.

Métricas de Rendimiento en Tiempo Real

El sistema calcula:

FPS

Indica cuántos cuadros por segundo procesa el sistema.

Fórmula:

$$FPS = \frac{1}{t_{actual} - t_{anterior}}$$

Tiempo de Reconocimiento

Mide el tiempo requerido para identificar una persona.

Fórmula:

$$T_{reconocimiento} = t_{fin} - t_{inicio}$$

5. Entrenamiento del Modelo

¿Por qué no se realizó un entrenamiento?

A diferencia de los modelos tradicionales de aprendizaje automático, en este proyecto no fue necesario entrenar una red neuronal desde cero.

El sistema utiliza el modelo preentrenado FaceNet, incorporado dentro de la biblioteca DeepFace. FaceNet fue entrenado previamente por Google utilizando millones de imágenes faciales, aprendiendo a representar cada rostro mediante vectores numéricos denominados *embeddings*.

Debido a que FaceNet ya posee la capacidad de extraer características faciales relevantes, el sistema únicamente utiliza dichas características para comparar rostros nuevos contra una base de datos de imágenes conocidas.

Por lo tanto, el proceso realizado corresponde a una fase de inferencia, no de entrenamiento.

Funcionamiento del Reconocimiento

Cuando una imagen es capturada por la cámara, el sistema sigue los siguientes pasos:

1. Detecta el rostro mediante OpenCV.
2. Extrae las características faciales usando FaceNet.
3. Genera un embedding facial.
4. Compara ese embedding con los embeddings de las imágenes almacenadas en la base de datos.
5. Calcula la similitud mediante la métrica de distancia coseno.
6. Selecciona la coincidencia con menor distancia como resultado de reconocimiento.

El proceso puede representarse como:

Imagen → Detección Facial → FaceNet

↓

Embedding Facial

↓

Comparación con Base de Datos

↓

Persona Identificada

Construcción de la Base de Conocimiento

Aunque no se realizó entrenamiento, fue necesario construir una base de datos facial.

Para ello se siguieron las siguientes etapas:

1. Recolección de imágenes

Se capturaron imágenes de cada estudiante.

Ejemplo:

```
bd/  
├── Jorge  
├── Miguel  
└── Karim
```

2. Aumento de Datos

Mediante Albumentations se generaron cinco variaciones por cada imagen original.

Las transformaciones incluyeron:

- Rotación.
- Cambio de brillo y contraste.
- Volteo horizontal.
- Ruido gaussiano.
- Desenfoque.

Esto permitió aumentar la diversidad visual del conjunto de imágenes.

3. Indexación Facial

La primera vez que DeepFace ejecuta `find()`, genera internamente los embeddings de todas las imágenes del directorio `bd`.

Estos embeddings se almacenan en caché para acelerar futuras búsquedas.

Justificación de la Elección de FaceNet

Se eligió FaceNet debido a:

- Alta precisión en reconocimiento facial.
- Modelo ampliamente validado en investigación.
- Capacidad para trabajar con pocos ejemplos por persona.
- Integración directa con DeepFace.
- Compatibilidad con CPU y GPU.
- No requiere reentrenamiento para agregar nuevos usuarios.

Ventaja principal

La incorporación de un nuevo estudiante únicamente requiere agregar sus imágenes a la carpeta correspondiente, sin necesidad de volver a entrenar el modelo completo.

6. Evaluación del Modelo

Archivo:

`evaluar_modelo.py`

Se utilizó un conjunto de prueba independiente almacenado en:

`test/`

Cada carpeta contiene imágenes pertenecientes a una persona específica.

Ejemplo:

`test/`

- |— Jorge
- |— Miguel
- |— Karim

Para cada imagen:

1. Se ejecuta `DeepFace.find()`.
2. Se obtiene la identidad predicha.
3. Se compara contra la identidad real.
4. Se almacenan resultados en:

`y_true`

`y_pred`

Métricas Utilizadas

Accuracy

Mide el porcentaje total de predicciones correctas.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Mide cuántas de las predicciones positivas fueron correctas.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Mide la capacidad del sistema para encontrar correctamente los casos positivos.

$$\text{Recall} = \frac{TP}{TP + FN}$$

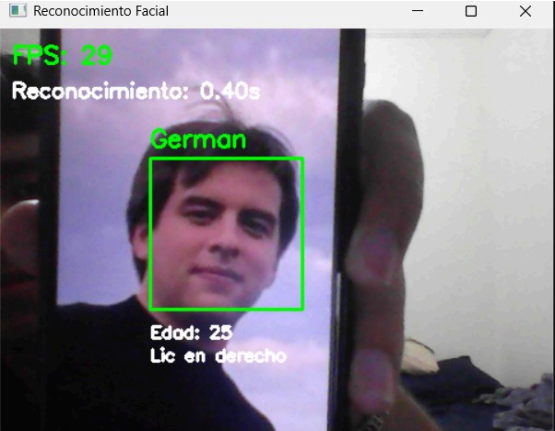
F1-Score

Combina Precision y Recall en una sola métrica.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

7. Resultados

Durante las pruebas se obtuvieron los siguientes resultados



```
Real: German -> Predicción: German
Real: German -> Predicción: German
Real: Jorge -> Predicción: Jorge
Real: Jorge -> Predicción: Flor
Real: Karim -> Predicción: Karim
Real: Miguel -> Predicción: Miguel
Real: Miguel -> Predicción: Miguel
Real: Valeria -> Predicción: Valeria

===== RESULTADOS =====
Accuracy : 0.8750
Precision: 1.0000
Recall   : 0.8750
F1 Score : 0.9167
```

===== REPORTE =====				
	precision	recall	f1-score	support
Flor	0.00	0.00	0.00	0
German	1.00	1.00	1.00	2
Jorge	1.00	0.50	0.67	2
Karim	1.00	1.00	1.00	1
Miguel	1.00	1.00	1.00	2
Valeria	1.00	1.00	1.00	1
accuracy			0.88	8
macro avg	0.83	0.75	0.78	8
weighted avg	1.00	0.88	0.92	8

Nota: en el caso de flor y los demás sujetos no se tenían fotos para el testing, por eso todo sale en 0, solo de los otros 5 sujetos se contaban con imagenes para testing

Interpretación:

- Accuracy cercano a 1 indica alta tasa de reconocimiento.
- Precision alta significa pocas identificaciones incorrectas.
- Recall alto indica que la mayoría de las personas fueron detectadas correctamente.
- F1-Score alto representa un equilibrio adecuado entre precisión y cobertura.